

多点标定

[toc]

项目简介

sample/multi_point_calibration文件夹下的示例展示了在windows和linux平台下，通过uart命令通路对机芯中NUC-T表进行修正，以解决客户整机在不同的环境温度下，对多个温度点进行准确测温的需求。

该示例展示了以下功能：

- NUC-T文件更新
- 点测温的环境变量温度修正

该示例从上层（应用层）到下层（通信协议层）分别依赖了：

接口适配层（interfaces文件夹）中的libircam、libircmd模块

驱动层（drivers文件夹）中的libiruart、libiruvic、libirdfu模块

其他动态库（other文件夹）中的libirupgrade、libirtemp模块

可执行文件构建步骤

1.选择合适的命令通信通路

根据当前机芯使用的命令通路，在multi_point_calibration/src/sample.cpp文件中进行设备节点的初始化和参数配置，目前示例支持的命令通路为UART命令通路：

1.1.UART命令通路

- 支持uart命令通路的平台类型：windows和linux
 - 在windows平台下：
 - 可执行文件编译依赖库：libircam、libircmd、libiruart、libirupgrade、libirtemp
 - 可执行文件运行依赖库：libircam、libircmd、libiruart、libirupgrade、libirtemp
 - 设备节点创建：在`multi\_point\_calibration/src/sample.cpp`文件创建uart_handle

```
IruartHandle_t* uart_handle = NULL;
uart_handle = iruart_handle_create(ir_control_handle);
```
 - 设备节点参数赋值：在multi_point_calibration/src/sample.cpp文件中设置需要打开的uart串口编号

```
char dev_info[] = "UART:\\\\.\\COM16;";    // 需要打开的设备节点为COM16
```
 - 在linux平台下：
 - 可执行文件编译依赖库：libircam、libircmd、libiruart、libirupgrade、libirtemp
 - 可执行文件运行依赖库：libircam、libircmd、libiruart、libirupgrade、libirtemp
 - 设备节点创建：在multi_point_calibration/src/sample.cpp文件创建uart_handle

```
IruartHandle_t* uart_handle = NULL;
uart_handle = iruart_handle_create(ir_control_handle);
```
 - 设备节点参数赋值：在multi_point_calibration/src/sample.cpp文件中设置需要打开的uart串口编号

```
char dev_info[] = "/dev/ttyUSB0;";    // 需要打开的设备节点为/dev/ttyUSB0
```
- 设备波特率设置
 - 在调用adv_device_open函数后调用adv_device_init函数来设置波特率
 - 对于没有调用adv_device_init函数设置波特率的情况，SDK内部将默认使用115200波特率与机芯交互

2. 程序编译

2.1 windows平台

使用Visual Studio进行编译

1. 参照**1.选择合适的命令通信通路**说明内容，将所选择的命令通路的**可执行文件编译依赖库**的导入库放入multi_point_calibration/libs/Release/x64/dll文件夹下
 - 以uart为例，需要将libircam.lib、libircmd.lib、libiruart.lib、libirupgrade.lib、libirtemp.lib放入`multi\_point\_calibration/libs/Release/x64/dll`文件夹下
2. 进入msvc文件夹下，双击打开.sln文件
3. 配置项目属性，在链接器的附加依赖项中填入所有的**可执行文件编译依赖库**的导入库
 - 以uart为例，填写内容如下

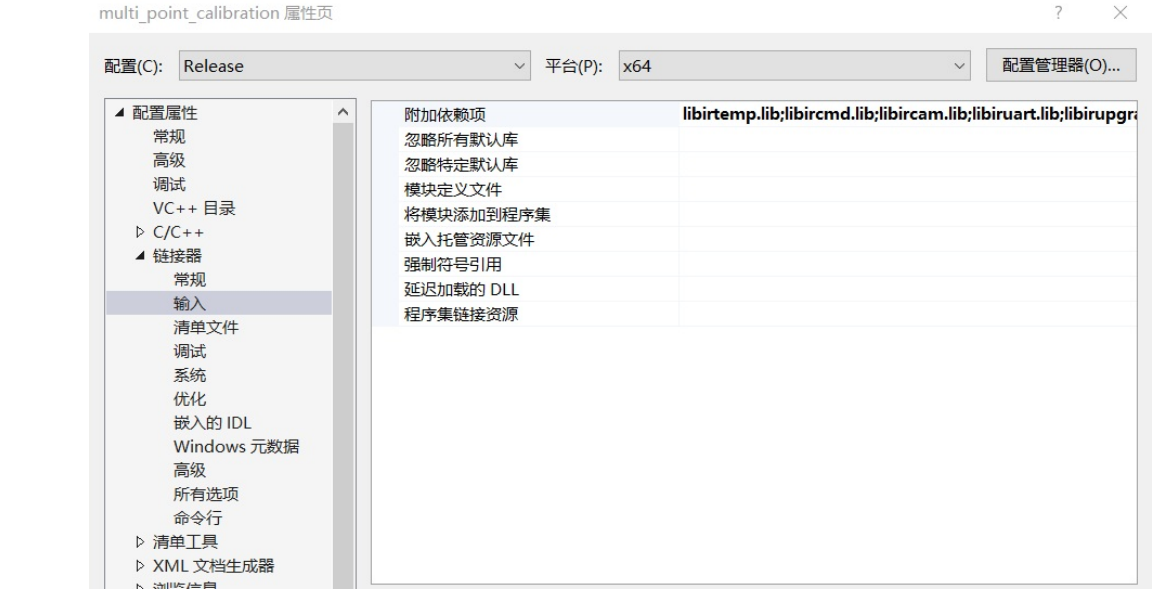


image-20230607112611546

- 勾选合适的编译环境（目前仅支持release|x64编译环境）进行编译，生成可执行程序
- 参照1.选择合适的命令通信通路说明内容，将所选择的命令通路的可执行文件编译依赖库的动态库放入multi_point_calibration/Release/x64文件夹下
 - 以uart为例，需要将libircam.dll\libircmd.dll\libiruart.dll\libirupgrade.dll\libirpack_too.dll放入multi_point_calibration/Release/x64/dll文件夹下

2.2 linux平台

- 参照1.选择合适的命令通信通路说明内容，将所选择的命令通路的可执行文件运行依赖库的共享库放入sample文件夹同级目录的drivers文件夹中
- 修改multi_point_calibration/CMakeList.txt中target_link_libraries中的依赖库链接配置，加上所有的可执行文件编译依赖库的共享库
 - 以uart通路为例，相关链接配置应修改如下

```
target_link_libraries(sample ircam ircmd iruart irupgrade irtemp -lm)
```

- 返回到sample/multi_point_calibration目录下，创建build`文件夹
- 进入build文件夹，使用如下命令即可生成可执行文件sample

```
camke ..
make
```

功能展示

运行sample.exe后，会出现下图所示提示,根据提示选择对应增益。

```
-----please choose the number of gain mode-----
0:low gain
1:high gain
-----
```

image-20230607114335032

低增益

```
-----please choose the number of gain mode-----
0:low gain
1:high gain
-----
0
请输入参与多点标定的点的数量:
```

image-20230607114834672

- 键入0选择低增益
- 在请输入参与多点标定的点的数量提示后键入参与多点标定的点的数量
- 当看到如下提示时说明更新成功

```
multi point calibration sample finish
```

image-20230607115121679

高增益

```
-----please choose the number of gain mode-----
0:low gain
1:high gain
-----
1
请输入参与多点标定的点的数量:
```

image-20230607115248724

- 键入1选择高增益
- 在请输入参与多点标定的点的数量提示后键入参与多点标定的点的数量
- 当看到如下提示时说明更新成功
- multi point calibration sample finish

具体软件实现步骤：

在sample.h中使用宏区分不同的功能。

image-20231023122839395

```
#define NUC_TABLE 0
#define CORRECT_TABLE 1
#define CORRECT_TABLE_SIZE 4088

#define MULTI_POINT_CALIBRATION
// #define ENHANCE_DISTANCE_TEMP_CORRECT
```

使用多点标定功能则打开 MULTI_POINT_CALIBRATION

使用环境温度修正功能则打开 ENHANCE_DISTANCE_TEMP_CORRECT

多点标定

1.数据采集，需要根据标定温度点下采集整机状态下模组正对黑体的温度值。注意需要打快门（basic_ffc_update）后采集10-15帧的图像数据，并取画面中心温度的均值（如初始化中multi_point_temp中4722 / 16.0 - 273.15，5602 / 16.0 - 273.15等，以机芯实际输出的温度值为准）作为多点标定的该温度点下的实测温度。同时用测温枪测得黑体的真实温度，作为多点二次标定该温度点下的设定温度（如初始化中multi_point_temp中的20，80，120，140，以实际采集的温度值为准）。该示例以高增益下的多点标定为例，共四个温度点。目前可以按照厂内标定黑体温度设置（高增益：15℃，40℃，150℃，低增益：15℃，40℃，150℃，300℃）

2.初始化部分

```
//设定四个点的环境变量修正参数，例如如下设置：
float ems[4] = { 1, 1, 0.95, 1 };
float hum[4] = { 0.45, 0.45, 0.45, 0.45 };
float dist[4] = { 0.25, 3, 3, 4 };
float ta[4] = { 25, 25, 25, 25 };
float tu[4] = { 25, 25, 25, 25 };
EnvCorrectParam env_cor_param[] = {
    { dist[0], ems[0], hum[0], ta[0], tu[0] }, \
    { dist[1], ems[1], hum[1], ta[1], tu[1] }, \
    { dist[2], ems[2], hum[2], ta[2], tu[2] }, \
    { dist[3], ems[3], hum[3], ta[3], tu[3] } };
//初始化多点标定的参数
MultiPointCalibNuc_t multi_point_nuc[] = { { 0, 0 }, { 0, 0 }, { 0, 0 }, { 0, 0 } };
MultiPointCalibTemp_t multi_point_temp[] = { { 4722 / 16.0 - 273.15, 20 }, \
{ 5602 / 16.0 - 273.15, 80 }, { 6122 / 16.0 - 273.15, 120 }, { 6500 / 16.0 - 273.15, 140 } };
MultiPointCalibArray_version3_t multi_point_calib_array;
//定义三个数组，分别存储从机芯读取的数据和要修正后的数据
uint16_t org_nuc_table[ADVANCED_NUC_T_SIZE] = { 0 };
uint16_t new_nuc_table[ADVANCED_NUC_T_SIZE] = { 0 };
uint16_t correct_table[CORRECT_TABLE_SIZE] = { 0 };
//确定参与多点标定的点数
MultiPointCalibInputParam_version3_t multi_point_calib_input_param;
printf("请输入参与多点标定的点的数量:");
scanf("%d", &multi_point_calib_input_param.multi_point_num);
//给多点标定参数申请空间
multi_point_calib_input_param.multi_point_param = (AdvancedMultiPointCalibParam_t*)malloc(sizeof(AdvancedMultiPointCalibParam_t));
multi_point_calib_input_param.multi_point_calib_array = (MultiPointCalibArray_version3_t*)malloc(sizeof(MultiPointCalibArray_version3_t));
```

3.让机芯默认使用default区域的NUC_T数据

```
//默认使用default区域NUC_T数据
ret = adv_second_calib_flag_set(ircmd_handle, ADV_SECOND_CALI_FLAG_NUC_T, ADV_SECOND_CALI_DISABLE);
```

4.从机芯中default区域读NUC_T数据

```
//根据高低增益模式(gain mode)不同,调用read_file从机芯中读取不同增益对应的原始nuc表，存储到org_nuc_table.
ret = read_file(ircmd_handle, gain_mode, NUC_T, org_nuc_table);
```

```
//根据高低增益模式(gain mode)不同,调用read_file从机芯中读取不同增益对应的距离修正表，存储到correct_table
ret = read_file(ircmd_handle, gain_mode, CORRECT, correct_table);
```

```
//如果不使用机芯中距离修正表的数据，我们也可以采用读取现有距离修正表的方式获取距离修正表
fp = fopen("RS300_9.1mm锃窗_Ver259_H.bin", "rb");
```

5.计算每个点经过NUC_T反查表后的NUC值

```
for (int i = 0; i < multi_point_calib_input_param.multi_point_num; i++)
{
    multi_point_calib_input_param.multi_point_param->high_temp_env_cor_param = env_cor_param[i];
    multi_point_calib_input_param.multi_point_param->low_temp_env_cor_param = env_cor_param[i];
    multi_point_calib_input_param.multi_point_temp = &multi_point_temp[i];
    multi_point_calib_input_param.multi_point_nuc = &multi_point_nuc[i];
    //实测温度(output_nuc)NUC_T反算
    ret = reverse_calc_NUC_with_nuc_t_table_len(org_nuc_table, multi_point_calib_input_param.multi_point_temp->output_temp, \
        ADVANCED_NUC_T_SIZE, &multi_point_calib_input_param.multi_point_nuc->output_nuc);

    //对设定温度(setting_temp)进行环境变量反算
    ret = reverse_enhance_distance_temp_correct(&(multi_point_calib_input_param.multi_point_param->high_temp_env_cor_param), \
        correct_table, CORRECT_TABLE_SIZE, multi_point_calib_input_param.multi_point_temp->setting_temp, \
        &multi_point_calib_input_param.multi_point_temp->setting_temp);

    //设定温度(setting_temp)NUC_T反算
    ret = reverse_calc_NUC_with_nuc_t_table_len(org_nuc_table, multi_point_calib_input_param.multi_point_temp->setting_temp, \
        ADVANCED_NUC_T_SIZE, &multi_point_calib_input_param.multi_point_nuc->setting_nuc);
}
```

6.用步骤5中计算得到的NUC值来计算新的NUC_T表

```
//将5中得到NUC值存入多点标定输入参数
multi_point_calib_input_param.multi_point_nuc = multi_point_nuc;
//计算新的NUC-T表
multi_point_calc_new_nuc_table_with_version(&multi_point_calib_input_param, MULTI_POINT_CALIB_V3, new_nuc_table);
```

7.将新NUC_T表写入机芯中二次标定数据区域的NUC_T数据区

```
ret = download_nuc_file(ircmd_handle, gain_mode, dev_param, new_nuc_table);
```

8.将二次标定区域的NUC_T数据标志位置起，机芯默认使用二次标定区域的NUC_T数据

```
adv_second_calib_flag_set(ircmd_handle, ADV_SECOND_CALI_FLAG_NUC_T, ADV_SECOND_CALI_ENABLE);
```

9.释放步骤2中申请的空间

```
free(multi_point_calib_input_param.multi_point_param);
free(multi_point_calib_input_param.multi_point_calib_array);
```

环境变量温度修正

1.初始化部分

```
//设定环境变量修正参数，参考多点标定，可按需求更改
float ems[4] = { 1, 1, 0.95, 1 };
float hum[4] = { 0.45, 0.45, 0.45, 0.45 };
float dist[4] = { 0.25, 3, 3, 4 };
float ta[4] = { 25, 25, 25, 25 };
float tu[4] = { 25, 25, 25, 25 };
EnvCorrectParam env_cor_param[] = {
    { dist[0], ems[0], hum[0], ta[0], tu[0] }, \
    { dist[1], ems[1], hum[1], ta[1], tu[1] }, \
    { dist[2], ems[2], hum[2], ta[2], tu[2] }, \
    { dist[3], ems[3], hum[3], ta[3], tu[3] } };

//输入测温目标点的坐标
IrcmdPoint_t point_pos;
float org_temp = 0;
float new_temp = 0;
uint32_t table_len = 0;

printf("Please enter point coordinate\n");
scanf("%hd %hd", &(point_pos.x), &(point_pos.y));
```

2.从机芯中读取距离修正表

```
ret = read_file(ircmd_handle, gain_mode, CORRECT, correct_table);
if (ret != 0)
{
    printf("fail to read single correct file\n");
    return ret;
}
```

3.发命令获取机芯测得的目标点的原始温度

```
ret = basic_point_temp_info_get(ircmd_handle, point_pos, &org_temp);
if (ret != IRLIB_SUCCESS)
{
    printf("fail to get point temp\n");
    return ret;
}
```

4.环境变量温度修正

```
ret = enhance_distance_temp_correct(&env_cor_param[0], correct_table, table_len, org_temp, &new_temp);
if (ret != IRLIB_SUCCESS)
{
    printf("fail to enhance_distance_temp_correct\n");
    return ret;
}
printf("point_temp_value is %f\n", new_temp);
```

注意事项

- 1. 多点标定的目标为生成新的NUC_T表写入机芯用于测温模块，写入的新NUC_T表存放在二次标定区域的NUC_T数据区域，故用户在使用多点标定功能并将新的NUC_T表写入机芯后，只需要将二次标定数据区域的NUC_T数据标志位置起，机芯就会使用二次标定数据区域的新NUC_T表。
- 2. 如果因出现异常或其他原因导致二次标定结果不准确，可按照sample的流程从第1步开始重新进行二次标定。
- 3. 标定距离：实际使用距离测试最佳，建议在5m 或 5m 之外，如 13m 进行标定。
- 4. 目前的理论和其他实测结论是，目前软件和算法是可以支持任意多个黑体进行标定，温度点越多，温度点设置越平均，全温段测温精度越高。
- 5. 理论上，测试点温度距离标定点越近，测温精度越高。
- 6. 首先使用者应知道机芯当前所处增益模式，如果与预期增益模式不符，则自行切换到预期增益模式 在运行sample时，填写的增益模式一定要与机芯当前所处的增益模式对应